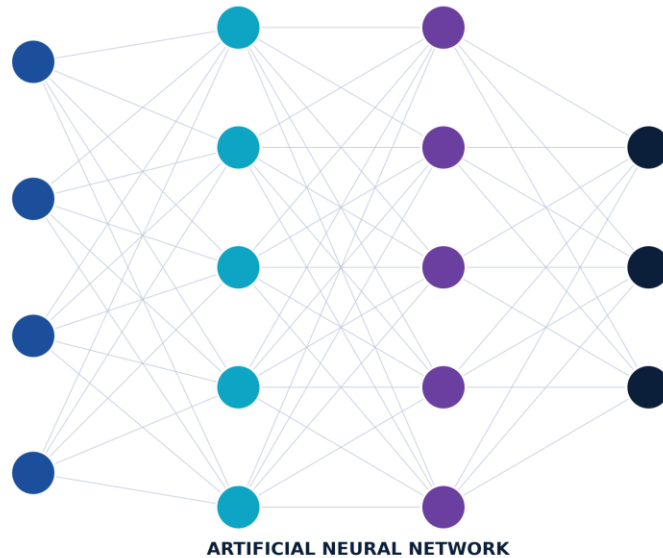




CO-2 AT-2

SCENARIO-BASED ASSESSMENT

ACTIVATION FUNCTIONS

Deep Neural Networks for Student Pass / Fail Classification

Student Name	Paleru Dharani Govardhan
Register Number	192525280
Department	Artificial Intelligence and Machine Learning
Course	Deep Learning
Assessment	CO-2 AT-2
Topic	Scenario-Based Assessment – Activation Functions

Submitted in partial fulfilment of the requirements of the Deep Learning course

Abstract

This report presents a scenario-based academic analysis of **activation functions** in deep neural networks, developed in the context of a college placement problem: classifying students as **Pass** or **Fail** using internal marks, attendance, and assignment marks as input features. The assessment investigates a $3 \rightarrow 4 \rightarrow 1$ neural network architecture and evaluates the behaviour of three widely used activation functions — Sigmoid, ReLU, and Leaky ReLU — within its hidden and output layers.

Two specific problems are examined. First, the use of **Sigmoid** in the hidden layer is shown to cause **slow learning** due to the **vanishing gradient problem**, where gradients shrink drastically for large positive or negative inputs, slowing weight updates during backpropagation. Second, the use of **ReLU** in the hidden layer is shown to occasionally cause the **dying ReLU problem**, in which certain neurons persistently output zero and stop contributing to learning.

The report recommends **Sigmoid** for the output layer, since it compresses the network output into the range 0 to 1, allowing it to be interpreted as the **probability that a student will pass**. For the hidden layer, **ReLU** is recommended as the primary choice due to its computational efficiency and strong gradient flow, with **Leaky ReLU** proposed as the corrective alternative whenever dying neurons are observed. Complete numerical calculations, comparative tables, and applied diagrams are provided to connect each theoretical concept to the student Pass/Fail prediction task.

Introduction

Educational institutions increasingly rely on data-driven methods to identify students who are at risk of failing a course, so that timely academic support can be provided. The scenario addressed in this report involves a college that wishes to build a **Deep Learning classification model** capable of predicting whether a student will **Pass** or **Fail**, based on three numerical indicators of academic performance: internal assessment marks, attendance percentage, and assignment marks.

A neural network is well suited to this task because the relationship between these three factors and the final outcome is unlikely to be purely linear — a student with moderate marks but excellent attendance and assignment performance may still pass, while a student with high marks but very poor attendance may not. Neural networks, through layers of neurons connected by weights and **non-linear activation functions**, can learn such complex, non-linear decision boundaries directly from data. The choice of activation function at each layer directly determines how effectively the network learns these relationships and how meaningfully its output can be interpreted.

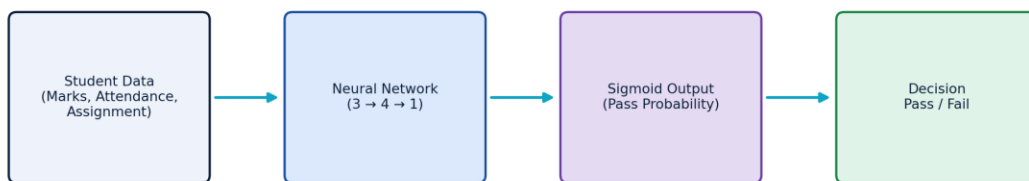


Figure 1: Conceptual Flow — Student Data → Neural Network → Pass Probability → Pass/Fail Decision

The remaining sections of this report formally define the problem, describe the neural network architecture, explain the theory of activation functions in depth, and apply that theory to resolve the two specific issues observed during testing — slow learning with Sigmoid and dead neurons with ReLU.

Problem Statement

A college wants to build a Deep Learning model that classifies students into **Pass** and **Fail** categories using three academic performance indicators. This can be formally stated as a **binary classification problem**:

Input Features (x)

- **x1** — Internal Marks (continuous, e.g. out of 100)
- **x2** — Attendance Percentage (continuous, 0–100%)
- **x3** — Assignment Marks (continuous, e.g. out of 100)

Output (y)

- **y** $\in \{0, 1\}$ — where 1 represents "Pass" and 0 represents "Fail", predicted as a probability $P(\text{Pass} \mid x_1, x_2, x_3) \in [0, 1]$

The neural network specified for this task has an **input layer of 3 neurons** (one per feature), a **hidden layer of 4 neurons** that learns intermediate feature representations, and an **output layer of 1 neuron** that produces the final pass probability. During testing, two issues were observed: slow convergence when Sigmoid was used in the hidden layer, and neurons permanently outputting zero when ReLU was used. This report analyses both issues and recommends a complete activation-function strategy for the network.

Objectives

The academic objectives of this scenario-based assessment are as follows:

1. **Understand activation functions** — their definition, purpose, and role in a neuron's computation.
2. **Analyse the limitations of Sigmoid** — particularly the vanishing gradient problem and its effect on learning speed.
3. **Understand ReLU** — its formula, advantages, and reasons for widespread use in hidden layers.
4. **Identify the dying ReLU problem** — its causes and consequences for network training.
5. **Understand Leaky ReLU** — as a corrective activation function for dead neurons.
6. **Select an appropriate output-layer activation** — for binary probability prediction.
7. **Apply activation functions mathematically** — through step-by-step numerical computation on a sample neuron.

Neural Network Architecture

The classification model uses a fully connected feed-forward architecture, labelled **3 → 4 → 1**, as illustrated below. Every neuron in one layer connects to every neuron in the next layer through a weighted connection, and each connection contributes to the weighted sum computed at the receiving neuron.

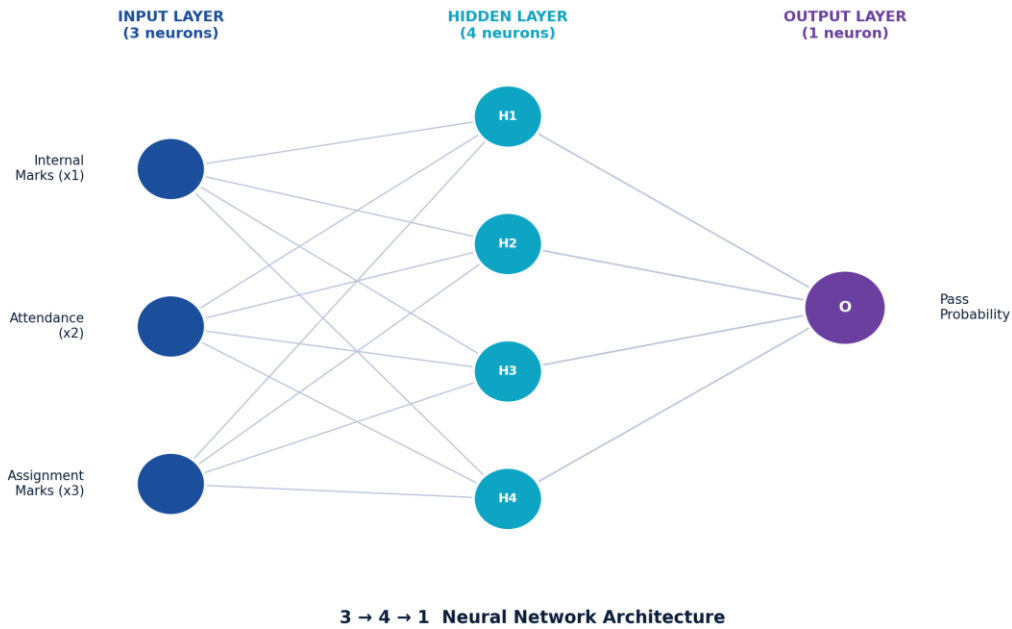


Figure 2: Detailed Architecture — 3 Input Neurons, 4 Hidden Neurons, 1 Output Neuron

Layer-by-Layer Description

- **Input Neurons (3):** Internal Marks, Attendance, and Assignment Marks are fed directly into the network without any transformation.
- **Weights (w):** Each connection between neurons carries a weight that scales the influence of the input on the receiving neuron. Weights are learned during training.
- **Bias (b):** An additional learnable constant added to the weighted sum, allowing the neuron's decision boundary to shift independently of the inputs.
- **Weighted Sum (z):** The linear combination of inputs, weights, and bias computed at each neuron before activation.
- **Activation Function (f):** A non-linear function applied to z to produce the neuron's final output.
- **Output (a):** The transformed value passed forward to the next layer, or, in the output layer, the final prediction.

Core Neuron Equations

Every neuron in the network performs two sequential steps — a linear weighted summation, followed by a non-linear activation:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$a = f(z)$$

Here, x_1, x_2, x_3 are the neuron's inputs, w_1, w_2, w_3 are the corresponding weights, b is the bias term, z is the weighted sum, $f(\cdot)$ is the chosen activation function, and a is the neuron's activated output. This two-step computation is repeated at every neuron in the hidden and output layers, allowing the network to progressively transform raw student data into a final pass/fail prediction.

What Is an Activation Function?

Definition

An **activation function** is a mathematical function applied to the weighted sum (z) of a neuron's inputs, producing the neuron's final output. It determines whether, and to what extent, a neuron should "fire" and pass information to the next layer.

Purpose of Activation Functions

- **Introduce non-linearity:** allowing the network to model complex, curved decision boundaries rather than only straight lines.
- **Enable deep representation learning:** stacking many linear layers without activation would collapse into a single linear transformation, regardless of depth.
- **Control output range:** certain activation functions (e.g., Sigmoid) constrain outputs to a specific interval, useful for probability estimation.
- **Support gradient-based learning:** activation functions must be differentiable so that gradients can propagate backward during training (backpropagation).

Why Non-Linearity Matters: Decision Boundaries

The Student Pass/Fail problem is unlikely to be perfectly separable by a straight line in three-dimensional feature space. Non-linear activation functions allow the hidden layer to bend and combine decision boundaries, enabling the network to correctly classify students whose outcome depends on a non-linear interaction between marks, attendance, and assignments.

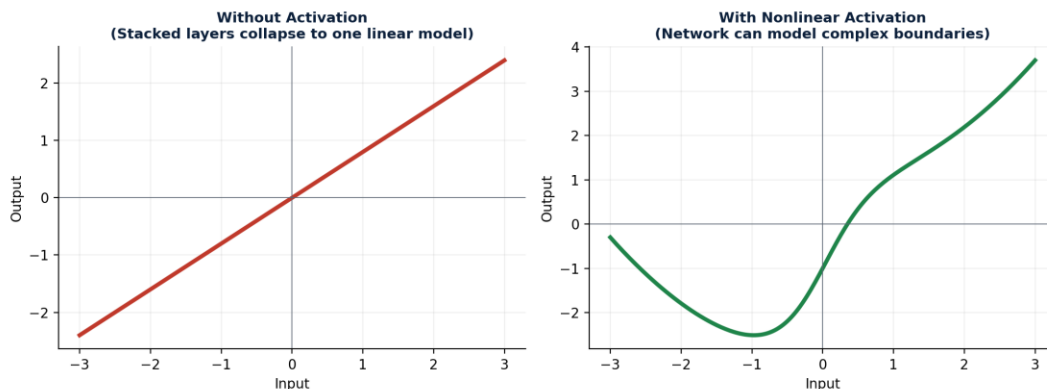


Figure 3: Linear Model (No Activation) vs Deep Non-Linear Model (With Activation)

Gradient Propagation

During training, the network learns by computing the **gradient** of the loss function with respect to each weight, using the chain rule of calculus (backpropagation). Because the activation function is part of this chain, its derivative directly affects how much each weight is updated. An activation function whose derivative becomes very small will cause very small weight updates — this is the root cause of the slow-learning issue explored in this report.

The Basic Neuron Computation

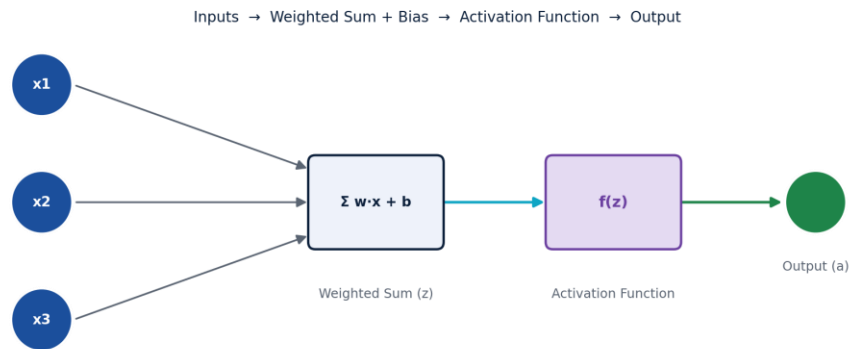


Figure 4: Artificial Neuron — Inputs, Weighted Sum, Activation, Output

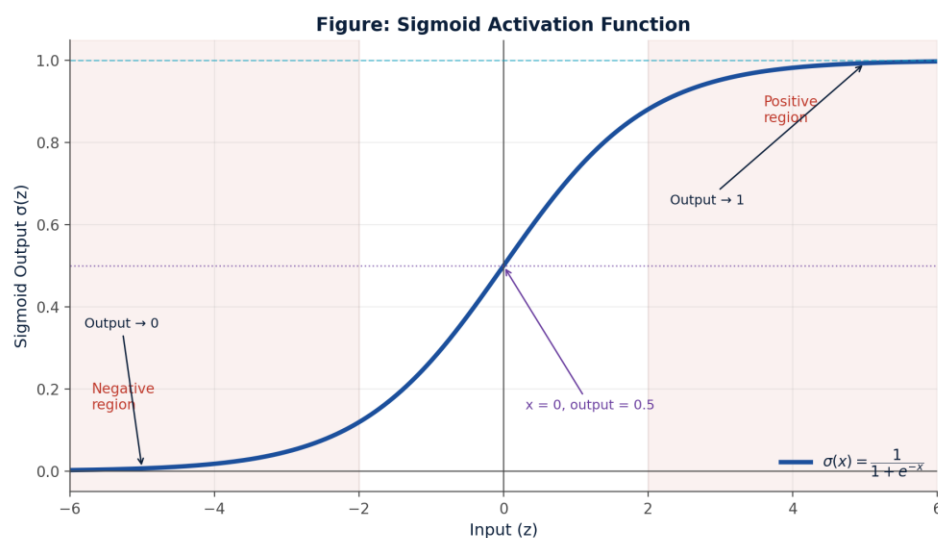
Sigmoid Activation Function

The **Sigmoid** (logistic) function is one of the earliest activation functions used in neural networks. It is defined as:

$$\sigma(x) = 1 / (1 + e^{-x})$$

Key Properties

- **Output range:** (0, 1) — the curve approaches but never reaches 0 or 1.
- **Shape:** a smooth, S-shaped (sigmoidal) curve, symmetric about $x = 0$, where $\sigma(0) = 0.5$.
- **Differentiability:** Sigmoid is smooth and differentiable everywhere, which is essential for gradient-based training.
- **Probability interpretation:** because its output always lies between 0 and 1, Sigmoid output can be directly interpreted as a probability.
- **Common use:** widely used in the output layer of binary classification models.

Figure 5: Sigmoid Activation Function, showing negative region, $x = 0$, and positive region

The Slow-Learning Problem: Vanishing Gradient

When Sigmoid is used in the **hidden layer**, the network in this scenario was observed to learn very slowly. The cause is the **vanishing gradient problem**:

The derivative of Sigmoid is $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$. This derivative reaches its **maximum value of only 0.25** at $x = 0$, and shrinks rapidly toward **0** as x moves toward large positive or large negative values, because the curve flattens ('saturates') in both directions.

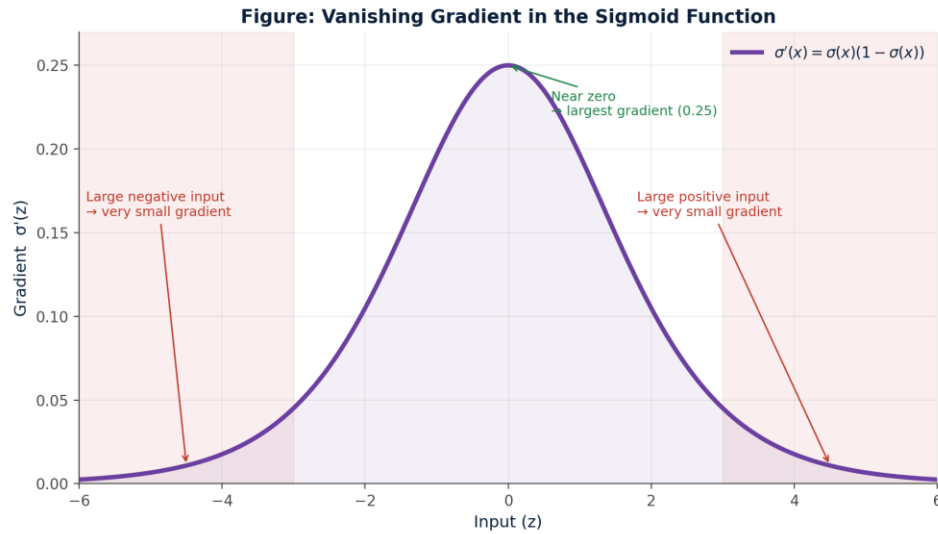


Figure 6: Vanishing Gradient — Sigmoid Derivative Approaches Zero at Extreme Inputs

During backpropagation, gradients are multiplied layer by layer. When several hidden layers each produce a small Sigmoid gradient, the overall gradient reaching the earlier layers becomes extremely small — sometimes near zero. As a result, weight updates in those layers become negligible, and **training slows down dramatically**, which directly explains the slow-learning behaviour observed in the scenario.

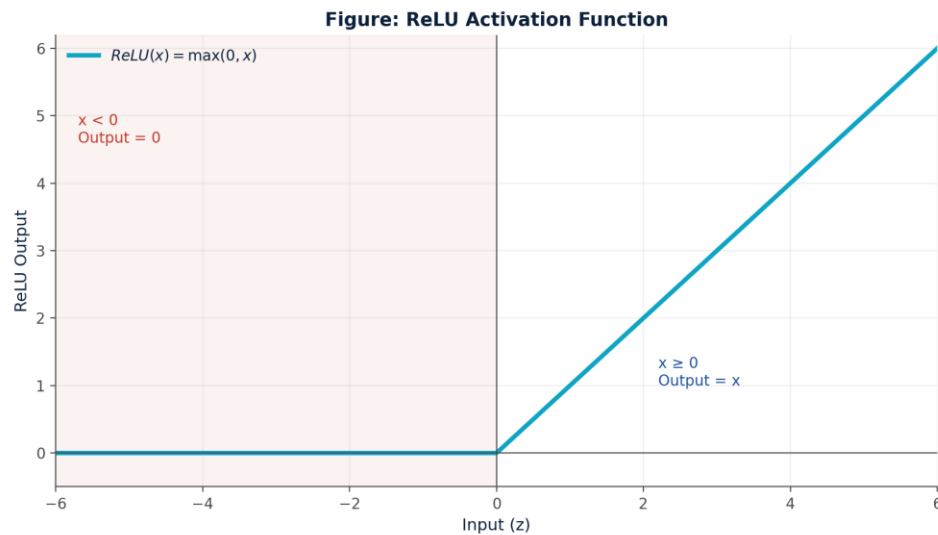
ReLU Activation Function

The **Rectified Linear Unit (ReLU)** is the most widely used activation function in modern hidden layers. It is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

Key Properties

- **Negative input ($x < 0$):** output is always 0.
- **Positive input ($x \geq 0$):** output equals the input value x , unchanged.
- **Computational simplicity:** ReLU requires only a comparison with zero, making it far cheaper to compute than exponential functions like Sigmoid.
- **Faster training:** because the gradient is exactly 1 for all positive inputs, there is no saturation on the positive side, allowing gradients to flow strongly through many layers.
- **Strong gradient for positive inputs:** this largely avoids the vanishing gradient problem for active neurons.
- **Widespread use:** ReLU's simplicity and efficiency make it the default choice for hidden layers in most modern deep networks.



Sigmoid vs ReLU — Comparative Summary

Property	Sigmoid	ReLU
Output Range	0 to 1	0 to ∞
Negative Input	Small positive value (never 0)	Exactly 0
Positive Input	Approaches 1 (always < 1)	Same as input value
Main Issue	Vanishing gradient	Dying ReLU
Typical Usage	Binary output layer	Hidden layers

Dying ReLU Problem

Definition

The **Dying ReLU Problem** occurs when a ReLU neuron continuously receives negative weighted-sum inputs. Because $\text{ReLU}(x) = 0$ for all $x < 0$, the neuron's output becomes **permanently 0**. Since the derivative of ReLU is also 0 for $x < 0$, **no gradient flows back** through that neuron during backpropagation — its weights are never updated again, and the neuron effectively "dies," contributing nothing to the network for the rest of training. This directly matches the second problem observed in the scenario, where several hidden-layer neurons using ReLU consistently output zero.

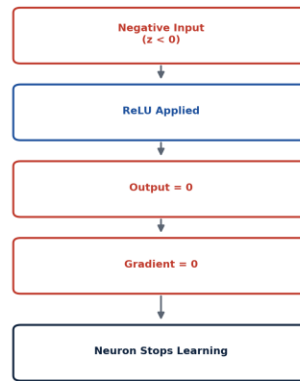


Figure 8: Dying ReLU Problem — From Negative Input to a Permanently Inactive Neuron

Common Causes

- **Large negative bias:** a strongly negative bias term shifts the weighted sum into negative territory for most inputs.
- **Large learning rate:** an aggressive weight update can push a neuron's weights into a region where it always produces negative sums.
- **Poor weight initialization:** weights initialized with unsuitable values can cause many neurons to start in, and remain in, the negative-output region.
- **Repeated negative inputs:** if the data distribution frequently produces negative weighted sums for a neuron, it may never recover a positive output.

Healthy vs Dying Neuron Behaviour



Figure 9: Comparison of a Healthy ReLU Neuron and a Dying ReLU Neuron

Leaky ReLU

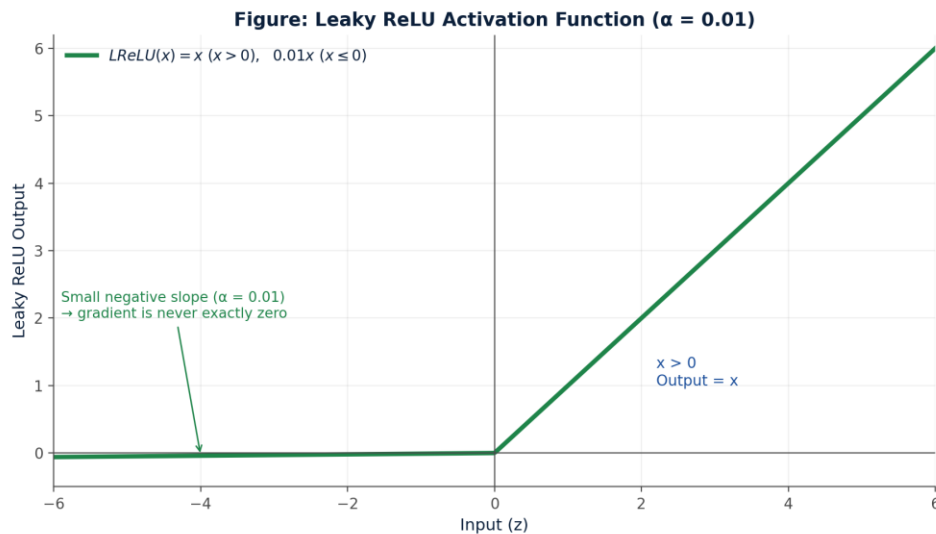
To overcome the dying ReLU problem, the **Leaky ReLU** activation function introduces a small, non-zero slope for negative inputs instead of flattening them to exactly zero. It is defined as:

$$\begin{aligned} \text{Leaky ReLU}(x) &= x, \text{ if } x > 0 \\ \text{Leaky ReLU}(x) &= \alpha x, \text{ if } x \leq 0 \end{aligned}$$

where α (alpha) is a small positive constant, commonly $\alpha = 0.01$, used throughout this report unless another value is explicitly specified.

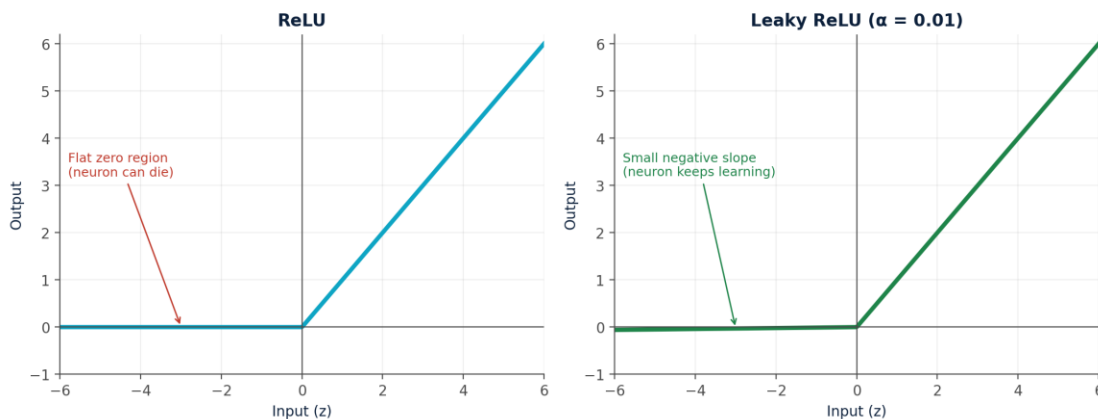
Key Properties

- **Positive region:** behaves exactly like standard ReLU — output equals input.
- **Negative region:** output is a small fraction (αx) of the input, rather than zero.
- **Prevents zero gradient:** the derivative for negative inputs is α (e.g., 0.01), not 0, so gradients never fully vanish.
- **Keeps neurons learning:** even neurons that frequently receive negative sums continue to receive small weight updates, preventing them from dying permanently.
- **Useful for negative inputs:** Leaky ReLU allows the network to retain a small amount of information from negative activations instead of discarding it entirely.



ReLU vs Leaky ReLU

Figure: Comparison of ReLU and Leaky ReLU for Negative Inputs



Output Layer Activation Selection

Question A: Which activation function should be recommended for the output layer? Explain why.

Recommended activation function: **Sigmoid**

Justification

The Student Pass/Fail model is a **binary classification problem** — the network must decide between exactly two outcomes, Pass or Fail. Sigmoid is the natural choice for the output layer in this setting because:

- Sigmoid produces an output strictly between **0 and 1**, matching the requirement that the output represent a probability.
- The output can be directly interpreted as **P(Pass | Internal Marks, Attendance, Assignment Marks)** — the probability that a given student will pass.
- Sigmoid is smooth and differentiable, which is required for the network to be trained using gradient descent with a binary cross-entropy loss function.
- Unlike ReLU, whose output is unbounded (0 to ∞), Sigmoid guarantees a bounded, probability-like output regardless of how large the incoming weighted sum becomes.

Example Interpretation

Suppose the output neuron produces:

Sigmoid Output = 0.82

This is interpreted as: **approximately 82% probability that the student will Pass**. A simple decision rule then converts this probability into a class label:

*If $P(\text{Pass}) \geq 0.5 \rightarrow \text{Predict "Pass"}$
Otherwise $\rightarrow \text{Predict "Fail"}$*

Why ReLU Is Unsuitable for the Output Layer

ReLU is unsuitable here because its output range is **0 to ∞** , not 0 to 1. A ReLU output of, say, 3.7 has no direct probabilistic meaning — it cannot be interpreted as "370% probability of passing." Using ReLU at the output would require an additional transformation to produce a valid probability, whereas Sigmoid provides this behaviour natively.



Figure 12: Output Layer Flow — Student Data → Network → Sigmoid → Probability 0.82 → PASS

Hidden Layer Activation Selection

Question B: Which activation function should be recommended for the hidden layer to overcome the slow-learning problem associated with Sigmoid?

Recommended activation function: **ReLU**

Why ReLU Is Preferred Over Sigmoid in Hidden Layers

- **Computationally simple:** requires only a $\max(0, x)$ comparison, unlike Sigmoid's exponential computation.
- **Reduces saturation:** positive inputs pass through unchanged, avoiding the flattening behaviour seen in Sigmoid's extremes.
- **Better gradient flow:** the derivative is exactly 1 for all positive inputs, so gradients do not shrink as they propagate backward.
- **Faster training:** stronger, more consistent gradients lead to quicker convergence than Sigmoid's small, saturating gradients.
- **Supports deep networks:** ReLU scales well to networks with many hidden layers, where vanishing gradients would otherwise compound.
- **Sparse activations:** neurons with negative input output exactly zero, producing a sparse, efficient internal representation.

Question C: What is the problem called when ReLU neurons continuously produce zero?

This is called the **Dying ReLU Problem** — described in detail in the previous section. It occurs when a neuron's weighted sum remains negative across most or all training examples, causing the neuron's output and gradient to remain permanently at zero.

Question D: Suggest one activation function that can overcome this problem.

The recommended solution is **Leaky ReLU** (with $\alpha = 0.01$), which replaces the flat zero region of standard ReLU with a small non-zero slope, ensuring that even neurons receiving negative inputs continue to receive a (small) gradient and remain capable of learning.

Recommended Decision Strategy for the Hidden Layer

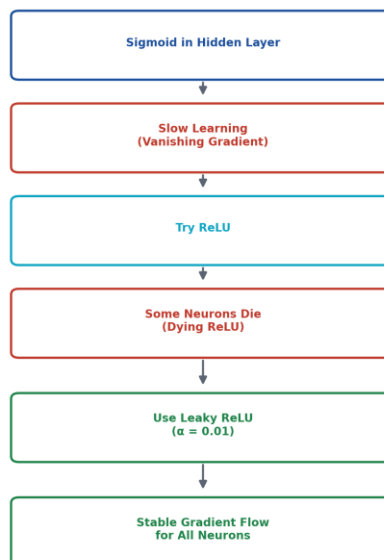


Figure 13: Decision Flow for Selecting the Hidden Layer Activation Function

Numerical Calculations

Question E: Calculate the output of a neuron using ReLU, Leaky ReLU, and Sigmoid.

The scenario provided in the assignment does not specify the exact numerical input or weighted-sum value for the neuron. To demonstrate the calculation method with full mathematical rigour, an **illustrative example** is used below, clearly labelled as such. The same step-by-step method applies directly to any specific value supplied in class.

Illustrative Example — Assume a hidden-layer neuron's weighted sum (after combining Internal Marks, Attendance, and Assignment Marks with their respective weights and bias) evaluates to:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b = -2$$

1. ReLU Calculation

$$\text{ReLU}(z) = \max(0, z)$$

Substituting $z = -2$:

$$\text{ReLU}(-2) = \max(0, -2) = 0$$

Result: ReLU output = **0** — since z is negative, the neuron produces zero output (this illustrates exactly how a dying-ReLU scenario begins).

2. Leaky ReLU Calculation ($\alpha = 0.01$)

$$\text{Leaky ReLU}(z) = \alpha z, \text{ for } z \leq 0$$

Substituting $z = -2$ and $\alpha = 0.01$:

$$\text{Leaky ReLU}(-2) = 0.01 \times (-2) = -0.02$$

Result: Leaky ReLU output = **-0.02** — a small negative value, instead of exactly zero.

3. Sigmoid Calculation

$$\sigma(z) = 1 / (1 + e^{-z})$$

Substituting $z = -2$:

$$\sigma(-2) = 1 / (1 + e^{-(-2)}) = 1 / (1 + e^2)$$

Since $e^2 \approx 7.389$:

$$\sigma(-2) = 1 / (1 + 7.389) = 1 / 8.389 \approx 0.1192$$

Result: Sigmoid output $\approx$ **0.1192** (approximately 11.92%).

Summary of Results ($z = -2$)

Activation Function	Formula	Calculated Output
ReLU	$\max(0, z)$	0.0000
Leaky ReLU ($\alpha = 0.01$)	αz	-0.0200
Sigmoid	$1 / (1 + e^{-z})$	0.1192

Comparative Analysis

Question F: Based on the calculated values, explain why Leaky ReLU can be useful compared with ReLU for negative inputs.

From the calculation above, for the negative input $z = -2$, standard ReLU produced an output of exactly **0**, while Leaky ReLU produced **-0.02**. Although both values are small, this difference is critical during training:

- With ReLU, the output and the gradient are both exactly zero, so **no learning signal** passes back through this neuron — its weights cannot be updated for this example.
- With Leaky ReLU, the output is a small but **non-zero** value, and its gradient ($\alpha = 0.01$) is also non-zero, so a small weight update **does** occur, keeping the neuron active in the learning process.
- Over many training iterations, this small but consistent gradient prevents the neuron from becoming permanently inactive — directly addressing the dying ReLU problem observed in the scenario.

Sigmoid vs ReLU vs Leaky ReLU — Full Comparison

Property	Sigmoid	ReLU	Leaky ReLU
Formula	$1/(1+e^{-x})$	$\max(0, x)$	$x \ (x > 0); \alpha x \ (x \leq 0)$
Output Range	$(0, 1)$	$[0, \infty)$	$(-\infty, \infty)$
Negative Input	Small positive (>0)	Exactly 0	Small negative (αx)
Positive Input	Approaches 1	Equals input	Equals input
Gradient Behaviour	Saturates, $\rightarrow 0$ at extremes	$1 \ (x > 0); 0 \ (x \leq 0)$	$1 \ (x > 0); \alpha \ (x \leq 0)$
Main Advantage	Probabilistic output	Fast, simple, strong gradient	Prevents dead neurons
Main Disadvantage	Vanishing gradient	Dying ReLU	Slightly more tuning (α)
Typical Application	Binary output layer	Hidden layers	Hidden layers (many dead neurons)
Effect on Learning	Can be slow in deep nets	Fast, may destabilise some neurons	Fast and stable
Hidden Layer Suitability	Limited (saturates)	Good (default choice)	Good (best if dying occurs)
Output Layer Suitability	Excellent (binary prob.)	Poor (unbounded)	Poor (unbounded)

Visual Comparison of All Three Curves

Figure: Side-by-Side Comparison of Sigmoid, ReLU and Leaky ReLU

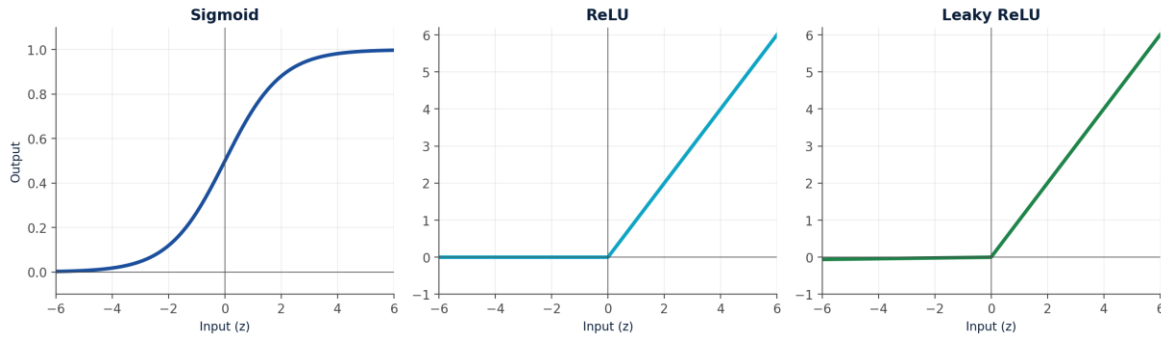


Figure 14: Sigmoid, ReLU, and Leaky ReLU Plotted Together for Direct Visual Comparison

Application to the Student Pass/Fail System

Combining the recommendations above, the complete Deep Learning workflow for the college's Pass/Fail prediction system proceeds through the following stages:

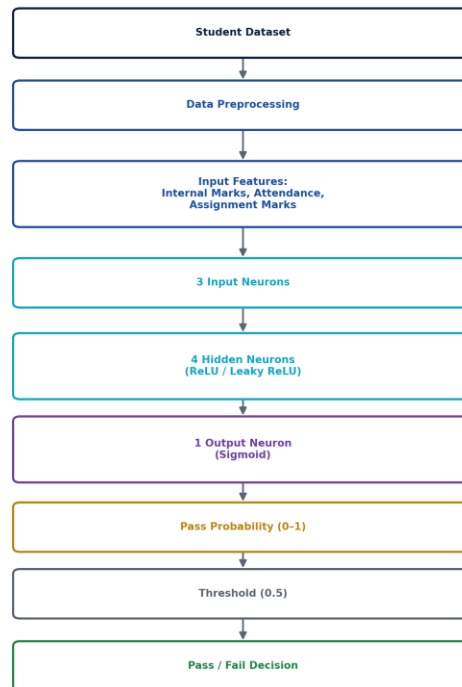


Figure 15: Complete Prediction Pipeline for the Student Pass/Fail Classification System

Stage-by-Stage Explanation

- **Student Dataset:** historical records of internal marks, attendance, and assignment marks, each labelled with the actual Pass/Fail outcome.
- **Data Preprocessing:** cleaning and normalising the three input features so they are on comparable numerical scales before training.
- **Input Layer (3 neurons):** receives the normalised Internal Marks, Attendance, and Assignment Marks.
- **Hidden Layer (4 neurons, ReLU / Leaky ReLU):** learns intermediate, non-linear combinations of the three features. ReLU is used by default; Leaky ReLU is substituted if dead neurons are detected during training.

- **Output Layer (1 neuron, Sigmoid):** compresses the final weighted sum into a probability between 0 and 1.
- **Threshold (0.5):** converts the probability into a discrete class label — Pass if ≥ 0.5 , Fail otherwise.

Illustrative Sample Predictions

The following table presents **illustrative** example predictions to demonstrate how the pipeline output would be interpreted. These are conceptual, hand-constructed values for demonstration only, not results from an experimentally trained model.

Student	Internal	Attendance	Assignment	Pass Probability	Prediction
A	85	90	88	0.94	Pass
B	45	60	50	0.18	Fail
C	72	80	75	0.81	Pass

Conclusion

- **Sigmoid** is the appropriate choice for the output layer, since it produces a bounded probability (0 to 1) representing the likelihood of a student passing.
- **ReLU** is generally the preferred choice for hidden layers due to its computational efficiency and strong gradient flow for positive inputs.
- **Sigmoid** can suffer from the vanishing gradient problem, causing slow learning when used extensively in hidden layers.
- **ReLU** can suffer from the dying ReLU problem, where neurons permanently output zero and stop learning.
- **Leaky ReLU** resolves this by allowing a small negative gradient ($\alpha = 0.01$), keeping neurons active even for negative inputs.
- Ultimately, activation-function selection **depends on the role of the layer** — hidden layers prioritise gradient flow and training speed, while the output layer prioritises meaningful, bounded interpretation of the result.

Learning Outcomes

This scenario-based assessment demonstrates the student's ability to:

1. **Select activation functions** appropriate to a given layer and task.
2. **Calculate activation outputs** manually, using ReLU, Leaky ReLU, and Sigmoid formulas.
3. **Identify vanishing-gradient problems** and explain their effect on training speed.
4. **Identify the dying ReLU problem** and its underlying causes.
5. **Explain Leaky ReLU** as a targeted correction for dead neurons.
6. **Apply activation-function theory** to a real-world academic classification problem.

References

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [2] Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. *Proc. 27th ICML*.
- [3] Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models. *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*.
- [4] Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep Sparse Rectifier Neural Networks. *Proc. 14th AISTATS*.

- [5] Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.
- [6] Stanford University CS231n Course Notes. *Convolutional Neural Networks for Visual Recognition — Activation Functions*. Stanford Computer Science Department.